GamePals: a Shared Control Framework for Video Game Accessibility

Matteo Manzoni
Dept. of Computer Science
Università degli Studi di Milano
Milano, Italy
matteo.manzoni@unimi.it

Filippo Corti
Dept. of Computer Science
Università degli Studi di Milano
Milan, Italy
filippo.corti1@studenti.unimi.it

Dragan Ahmetovic
Dept. of Computer Science
Università degli studi di Milano
Milano, Italy
dragan.ahmetovic@unimi.it

Sergio Mascetti
Dept. of Computer Science
Università degli Studi di Milano
Milano, Italy
sergio.mascetti@unimi.it

Abstract

Video games are usually played with hand-based controllers, creating accessibility barriers for players with upper-limb impairments. Existing assistive solutions, such as accessible controllers or alternative input modalities, may still be insufficient when many game actions have to be controlled rapidly and concurrently. This can be addressed through shared control of the game actions. In particular, human cooperation refers to shared control with another player, while partial automation delegates some game actions to a software agent, removing the need for human support. However, partial automation is available only in specific games and for specific tasks, like aim assistance, and no existing solution enables both human cooperation and partial automation in arbitrary video games. To address this problem, we propose GamePals, an open-source framework that enables shared control in arbitrary third-party games. The framework supports human cooperation out of the box and provides facilities to implement partial automation. We showcase the framework's capabilities in the Rocket League video game.

CCS Concepts

• **Applied computing** → *Computer games*; • **Human-centered computing** → **Accessibility technologies**; **Collaborative interaction**.

Keywords

Shared control; partial automation; human cooperation; accessible gaming.

ACM Reference Format:
Matteo Manzoni, Filippo Corti, Dragan Ahmetovic, and Sergio Mascetti. 2026. GamePals: a Shared Control Framework for Video Game Accessibility. In *The 28th International ACM SIGACCESS Conference on Computers and Accessibility (ASSETS '26), October 25–28, 2026, Vila Nova de Gaia, Portugal.* ACM, New York, NY, USA, 5 pages. https://doi.org/10.1145/3797867.3841212



This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License.
ASSETS '26, Vila Nova de Gaia, Portugal
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2521-0/26/10
https://doi.org/10.1145/3797867.3841212

1 Introduction

Many modern video games are played through hand-based interaction, in general using controller peripherals designed to be operated with both hands [12] and requiring players to perform multiple inputs simultaneously [7, 17, 28]. As a result, individuals with upper-limb impairments may face significant challenges when accessing video games [2, 5, 6, 12, 17, 28].

Various solutions have been proposed to improve video game accessibility for people with upper limb impairments, for example alternative controllers designed to be more accessible [17], or tools allowing to play through alternative interaction modalities such as head gestures [16, 22] or voice commands [3, 26]. However, these solutions may not be sufficient, for example, when the player must press multiple buttons simultaneously or within a very short time frame. In these cases, it is possible to delegate the control of some of the *game actions* to another player. This game modality, called human cooperation [2], is natively supported on *Windows* PCs, and *Xbox* consoles [19], and available through third-party solutions on other consoles [11]. In this setting, the player who receives assistance is referred to as the *pilot*, while the player providing assistance is called the *copilot*. However, one limitation of this approach is that it introduces a reliance on another person to access and experience the game, thereby constraining players' autonomy [2].

Partial automation solutions address this issue by substituting the human *copilot* with a software agent that assists the player in controlling *game actions* [2, 9]. However, partial automation is available only in a limited number of video games and is restricted to specific tasks, such as aim assistance in *Call of Duty* [1] or steering assistance in *Mario Kart* [13]. Previous research has investigated how partial automation can be applied to entire video games to increase their accessibility [8, 9, 18]. However, these solutions have been designed for ad hoc games, thus lacking generalizability to arbitrary third-party games and consequently limiting their adoption as broadly applicable accessibility solutions.

In this demo, we showcase *GamePals*, an open-source[1] framework that enables shared control in third-party video games, supporting both human cooperation and partial automation. Human

[1] Available at: https://lnk.ewlab.di.unimi.it/gamepals, under CC BY-NC 4.0 license [10].

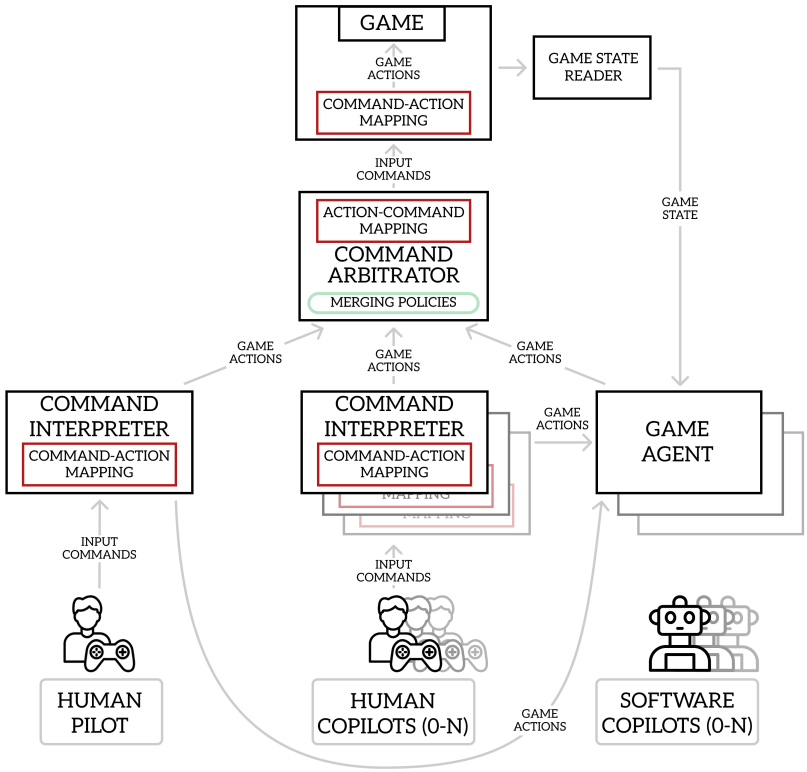


Figure 1: *GamePals* framework's architecture.

cooperation is fully supported by the framework for arbitrary third-party games out of the box, whereas partial automation requires defining game-specific software modules. In the following, we showcase the *GamePals* framework and how it can be adapted for a popular third-party video game – *Rocket League* [24] – to enable partial automation gameplay.

2 The *GamePals* Framework

Figure 1 shows *GamePals*'s architecture and its main components. In the following, we describe how *GamePals* can be configured to play third-party video games in human cooperation and partial automation.

2.1 Assigning Game Actions to the Players

To use *GamePals* with a third-party video game, it is first necessary to configure the set of *game actions* available for that game (*e.g.*, movement, jump, fire). Then, for each *game action*, it must be specified which human or software players control it. Each action can be assigned to one or more players. Thus, *GamePals* supports three different scenarios. One possibility is for every player to control all *game actions*. Alternatively, each action can be managed by a different player. Finally, some actions can be controlled exclusively by individual players, while others are shared among them. For example, Figure 2 illustrates a configuration in which the pilot controls three *game actions*, the copilot controls two, and one *game action* is jointly controlled by both the pilot and the copilot.

For human players, the action subdivision is handled by the *command interpreter* component (Figure 1), which intercepts player input from their physical controllers and translates it into the corresponding *game actions*. The mapping between inputs and *game actions* can be freely customized for each player. The *command interpreter* supports a wide variety of video game controllers, including the standard game controllers such as the *Xbox* controller [21] and accessibility devices such as the *Xbox Adaptive Controller* [20]. Note that, thanks to this approach, the *command interpreter* also allows each player to freely remap each *game action* to any input control on their device. This supports high personalization. For example, a player who can only use their left hand can remap all *game actions* to the left side of the game controller, while the human copilot can control the same actions using input commands on both sides of the controller. Software players' actions are instead managed by the *game agent* (Figure 1), which is a game-specific component responsible for generating *game actions*.

2.2 Combining Game Actions From Players

When a *game action* is shared among two or more players, a *merging policy* must be specified to determine how inputs are combined to control that *game action*. For example, if two players both control the movement, it is necessary to determine how to resolve conflicting directions. One possible *merging policy*, inspired by *Xbox Controller Assist* [19], is to sum the inputs of all players. In this case, if one player fully pushes the movement analog stick of their

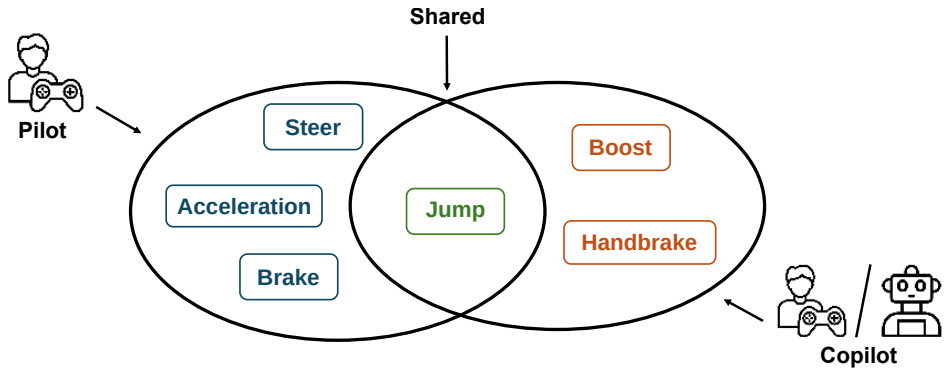


Figure 2: Diagram showing how game actions can be subdivided between the pilot and the copilot.

controller to the right while another fully pushes it to the left, the resulting direction sent to the game will be straight. *GamePals* allows the definition of arbitrary action-specific policies based on each player's inputs and role (*i.e.*, *pilot* or *copilot*).

The application of *merging policies* is handled by the *command arbitrator* component of the framework (Figure 1). This component receives the *game actions* executed by both human and software players and merges them into a single action based on the configured policies. The result is then translated into standard controller input and sent to the game via a *virtual controller* (which the game cannot distinguish from a real one).

2.3 Supporting Partial Automation

Partial automation requires implementing a *game adaptation* specific to each game to generate the software players' *game actions*. The *game adaptation* comprises two components: a *game state reader* and a *game agent* (Figure 1). The *game state reader* accesses the game's internal state, while the *game agent* receives the game state and the actions executed by human players to generate new *game actions*. One possible approach to accessing the game state is modding[2]. Alternative solutions, for example, include screen-based interpretation techniques [14] or direct memory reading [27].

Game agents can be implemented as rule-based systems that trigger predefined actions when specific game conditions are met. More advanced agents may rely on reinforcement learning to learn to play more complex video games, for which it is difficult to define explicit rules to guide their actions [25].

This demo presents a *game adaptation* that we developed for the *Rocket League* [24] video game (Section 3). The game state is accessed through modding, while the agent was developed with reinforcement learning.

3 Rocket League Game Adaptation

Rocket League [24] (Figure 3) is a team video game similar to soccer in which each player controls a vehicle and aims to score with a large ball into the opponent team's goal. The team that scores more goals by the end wins. Players control their vehicle through six actions: acceleration, brake, steer, jump, handbrake, and boost.

[2]Modding is "the act of changing a game, usually through computer programming, with software tools that are not part of the game" [23, p. 1250].



Figure 3: In-game screenshot from *Rocket League*.

The *game adaptation* reads the game state through modding, using a custom C/C++ plugin we developed with the *BakkesMod SDK* [4], which provides direct access to the game's internal memory data structures[3]. The plugin hooks into the main rendering loop, gathers the necessary data every frame (the game runs at 120 FPS), serializes it to JSON, and transmits it to the *game agent* via a local socket connection.

The *game agent* is based on *Nexto*[4], which is a state-of-the-art reinforcement learning bot for *Rocket League*. The bot receives the game state and outputs the optimal *game actions* to execute. However, *Nexto* was originally designed to control all *game actions* simultaneously. Thus, to integrate it into *GamePals*, we implemented a masking mechanism that filters out all *game actions* except those assigned to the software player in the configuration.

4 Demonstration

During the demo, participants will try *GamePals* while playing *Rocket League* (Figure 4). First, participants will select the shared control modality they would like to test. For example, they may play in human cooperation with another participant acting as *copilot*, they could play in partial automation, with the support of a software player, or they may combine the two modalities, with an arbitrary number of human *copilots* and supported by software *copilots*. Then, each participant will choose an input device between a standard *Xbox* controller [21] and an *Xbox Adaptive Controller* [20],

[3]Permitted in Rocket League in offline environments [15].
[4]https://github.com/Rolv-Arild/Necto



(a) The ball is heading past the goal. (b) Copilot boosts to reach the ball faster. (c) Pilot jumps to divert the ball into the goal.

Figure 4: Example of partial automation support using the configuration shown in Figure 5.

allowing them to experience how different input devices can be accommodated within the framework. Finally, participants will decide which *game actions* they want to control, which to delegate, and which commands to use for those actions.

For example, as shown in Figure 5, a player might control the steer, acceleration, brake, and jump *game actions*, while the *software copilot* controls boost, handbrake, and jump. Note that this example implements the configuration depicted in Figure 2 in which the control of the jump *game action* is shared between the pilot and the copilot. Figure 4 shows an example of the support provided with this configuration during gameplay.

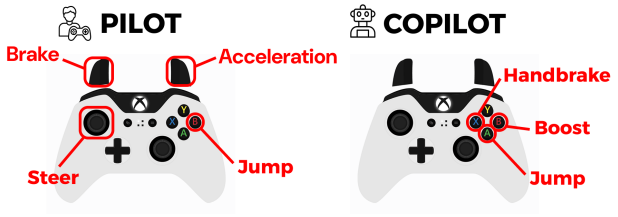


Figure 5: Example *game action* subdivision for *Rocket League*.

The demo will include predefined configurations, but participants will also be able to request that the supervisor create custom configurations to explore novel shared control approaches. Once *GamePals* has been configured, participants will play a match with the chosen configuration, lasting approximately 5 minutes. Based on our experience, introducing and configuring the system takes about 5 minutes. Thus, the total duration is about 10 minutes for each participant.

5 Conclusions and Future Work

GamePals is designed as a research probe that allows experimentation with all configurations of human cooperation and partial automation previously proposed in the literature – as well as novel ones – using existing third-party games. As future work, we will conduct user studies with *GamePals* to validate its ability to effectively support players with upper-limb impairments in accessing video games, and to understand how playing in shared control unfolds.

The adoption of *GamePals* as an assistive technology is limited by two main factors. First, the configuration requires editing a markup file. Therefore, as future work, we plan to develop a visual tool that allows end-users to edit the configuration. Second, only one *game adaptation* for partial automation is available. Therefore, we plan to develop *game adaptations* for other games.

As future work, we also aim to enhance communication between the player and the software agent. First, we will explore interaction mechanisms to allow players to directly influence the agent behavior during gameplay by communicating their needs and requests. Second, we will investigate how software agents can provide feedback about their actions to improve players' understanding of the game state and increase their sense of control over the game.

References

[1] Activision. 2003. Call of Duty. Video game series. https://www.callofduty.com
[2] Dragan Ahmeotivc, Matteo Manzoni, Filippo Corti, and Sergio Mascetti. 2026. Shared Control for Game Accessibility: Understanding Current Human Cooperation Practices to Inform the Design of Partial Automation Solutions. In *Proceedings of the 2026 CHI Conference on Human Factors in Computing Systems, to appear* (Barcelona) *(CHI '26)*. Association for Computing Machinery, New York, NY, USA.
[3] Dragan Ahmetovic, Daniele Riboli, Cristian Bernareggi, and Sergio Mascetti. 2021. RePlay: Touchscreen Interaction Substitution Method for Accessible Gaming. In *Proceedings of the 23rd International Conference on Mobile Human-Computer Interaction*. Association for Computing Machinery, New York, NY, USA, Article 27, 12 pages. doi:10.1145/3447526.3472044
[4] Andreas "bakkes" Bakke. n.d.. *BakkesMod Rocket League trainer*. https://www.bakkesmod.com
[5] Kevin Bierre, Jonathan Chetwynd, Barrie Ellis, D Michelle Hinn, Stephanie Ludi, and Thomas Westin. 2005. Game not over: Accessibility issues in video games. In *Proc. of the 3rd International Conference on Universal Access in Human-Computer Interaction*. Lawrence Erlbaum Associates (CRC Press), Mahwah, NJ, USA, 22–27.
[6] Mark Brown and Sky LaRell Anderson. 2021. Designing for disability: Evaluating the state of accessibility design in video games. *Games and Culture* 16, 6 (2021), 702–718.
[7] Michelle A Brown and I Scott MacKenzie. 2013. Evaluating video game controller usability as related to user hand size. In *Proceedings of the International Conference on Multimedia and Human Computer Interaction*. International ASET, Inc., Toronto, Ontario, Canada, 1–9.
[8] Jared E. Cechanowicz, Carl Gutwin, Scott Bateman, Regan Mandryk, and Ian Stavness. 2014. Improving player balancing in racing games. In *Proceedings of the First ACM SIGCHI Annual Symposium on Computer-Human Interaction in Play* (Toronto, Ontario, Canada) *(CHI PLAY '14)*. Association for Computing Machinery, New York, NY, USA, 47–56. doi:10.1145/2658537.2658701
[9] Gabriele Cimolino, Sussan Askari, and T.C. Nicholas Graham. 2021. The Role of Partial Automation in Increasing the Accessibility of Digital Games. *Proc. ACM Hum.-Comput. Interact.* 5, CHI PLAY, Article 266 (Oct. 2021), 30 pages. doi:10.1145/3474693
[10] Creative Commons. n.d.. *Attribution-NonCommercial 4.0 International*. https://creativecommons.org/licenses/by-nc/4.0/
[11] ConsoleTuner. n.d.. *Titan Two*. https://www.consoletuner.com/products/titan-two
[12] Mat Dalgleish. 2024. *Who Can Play? Rethinking Video Game Controllers and Accessibility*. Springer International Publishing, Cham, 43–71. doi:10.1007/978-3-031-34374-2_3

[13] Nintendo EAD. 2017. Mario Kart 8 Deluxe. Video game, released on Nintendo Switch.
[14] Kamolwan Kunanusont, Simon M. Lucas, and Diego Pérez-Liébana. 2017. General Video Game AI: Learning from screen capture. In *2017 IEEE Congress on Evolutionary Computation (CEC)*. 2078–2085. doi:10.1109/CEC.2017.7969556
[15] Rockt League. n.d.. *Code of Conduct.* https://www.rocketleague.com/code-of-conduct
[16] Matteo Manzoni, Dragan Ahmetovic, and Sergio Mascetti. 2024. Personalized Facial Gesture Recognition for Accessible Mobile Gaming. In *Computers Helping People with Special Needs*. Springer Nature Switzerland, Cham, 120–127. doi:10.1007/978-3-031-62846-7_15
[17] Jesse J Martinez, Jon E. Froehlich, and James Fogarty. 2024. Playing on Hard Mode: Accessibility, Difficulty and Joy in Video Game Adoption for Gamers with Disabilities. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems* (Honolulu, HI, USA) *(CHI '24)*. Association for Computing Machinery, New York, NY, USA, Article 524, 17 pages. doi:10.1145/3613904.3642804
[18] Lucas Medeiros and Flavio Coutinho. 2015. Developing an Accessible One-Switch Game for Motor Impaired Players. *Proceedings of SBGames* (2015), 236–239.
[19] Microsoft. 2017. *Xbox Controller Assist (formerly Xbox Copilot).* https://support.xbox.com/en-US/help/account-profile/accessibility/copilot
[20] Microsoft. n.d.. *Xbox Adaptive Controller.* https://www.xbox.com/accessories/controllers/xbox-adaptive-controller
[21] Microsoft. n.d.. *Xbox Controller.* https://support.xbox.com/en-US/help/hardware-network/controller/xbox-one-wireless-controller
[22] PlayAbility. n.d.. *PlayAbility.* https://www.playability.gg
[23] Nathaniel Poor. 2014. Computer game modders' motivations and sense of community: A mixed-methods approach. *New media & society* 16, 8 (2014), 1249–1267.
[24] Psyonix. 2015. Rocket League. Video game, released on PlayStation 4, Xbox One, Nintendo Switch, and PC.
[25] SIMA Team, Maria Abi Raad, Arun Ahuja, Catarina Barros, Frederic Besse, Andrew Bolt, Adrian Bolton, Bethanie Brownfield, Gavin Buttimore, Max Cant, Sarah Chakera, Stephanie C. Y. Chan, Jeff Clune, Adrian Collister, Vikki Copeman, Alex Cullum, Ishita Dasgupta, Dario de Cesare, Julia Di Trapani, Yani Donchev, Emma Dunleavy, Martin Engelcke, Ryan Faulkner, Frankie Garcia, Charles Gbadamosi, Zhitao Gong, Lucy Gonzales, Kshitij Gupta, Karol Gregor, Arne Olav Hallingstad, Tim Harley, Sam Haves, Felix Hill, Ed Hirst, Drew A. Hudson, Jony Hudson, Steph Hughes-Fitt, Danilo J. Rezende, Mimi Jasarevic, Laura Kampis, Rosemary Ke, Thomas Keck, Junkyung Kim, Oscar Knagg, Kavya Kopparapu, Rory Lawton, Andrew Lampinen, Shane Legg, Alexander Lerchner, Marjorie Limont, Yulan Liu, Maria Loks-Thompson, Joseph Marino, Kathryn Martin Cussons, Loic Matthey, Siobhan Mcloughlin, Piermaria Mendolicchio, Hamza Merzic, Anna Mitenkova, Alexandre Moufarek, Valeria Oliveira, Yanko Oliveira, Hannah Openshaw, Renke Pan, Aneesh Pappu, Alex Platonov, Ollie Purkiss, David Reichert, John Reid, Pierre Harvey Richemond, Tyson Roberts, Giles Ruscoe, Jaume Sanchez Elias, Tasha Sandars, Daniel P. Sawyer, Tim Scholtes, Guy Simmons, Daniel Slater, Hubert Soyer, Heiko Strathmann, Peter Stys, Allison C. Tam, Denis Teplyashin, Tayfun Terzi, Davide Vercelli, Bojan Vujatovic, Marcus Wainwright, Jane X. Wang, Zhengdong Wang, Daan Wierstra, Duncan Williams, Nathaniel Wong, Sarah York, and Nick Young. 2024. Scaling Instructable Agents Across Many Simulated Worlds. arXiv:2404.10179 [cs.RO] https://arxiv.org/abs/2404.10179
[26] VoiceAttack. n.d.. *VoiceAttack.* https://voiceattack.com
[27] How Bots Work. [n. d.]. Server-Side Bot Detection in Massive Multiplayer Online Games. ([n. d.]).
[28] Bei Yuan, Eelke Folmer, and Frederick C Harris Jr. 2011. Game accessibility: a survey. *Universal Access in the information Society* 10, 1 (2011), 81–100.